

EEE8089

M2M Technology IoT

Topic 3: M2M with SSH

Dr. Bystrov

School of Engineering
University of Newcastle upon Tyne

Aims/Objectives

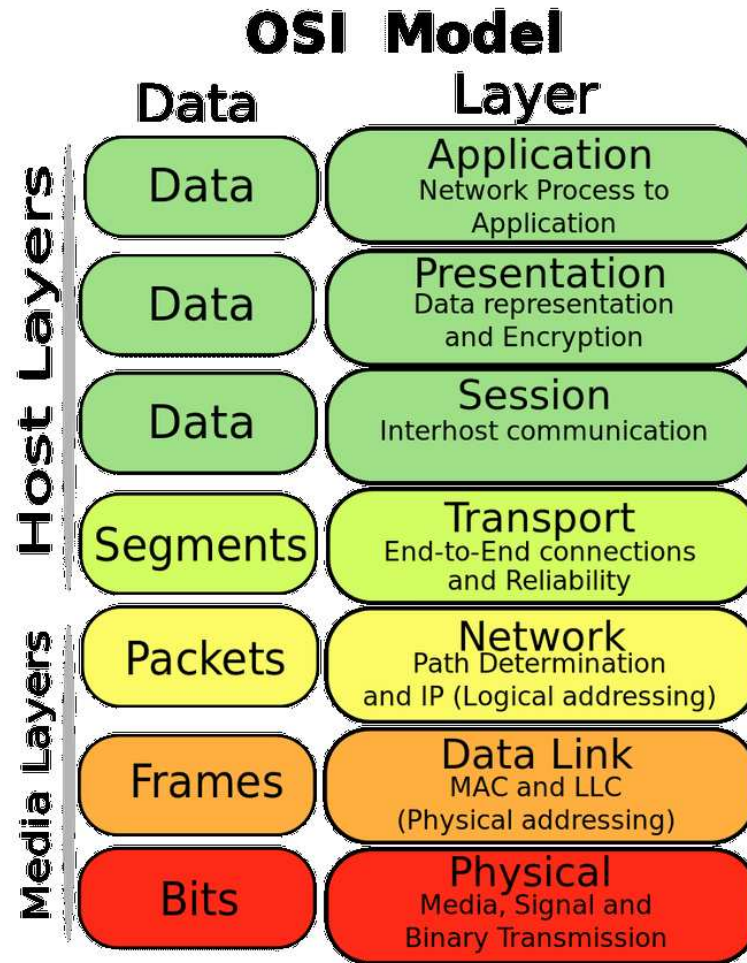
Aim

- Secure Shell
- M2M based on SSH

Objectives

- SSH background
- M2M Links
- M2M Connections
- Example

Open Systems Interconnection model

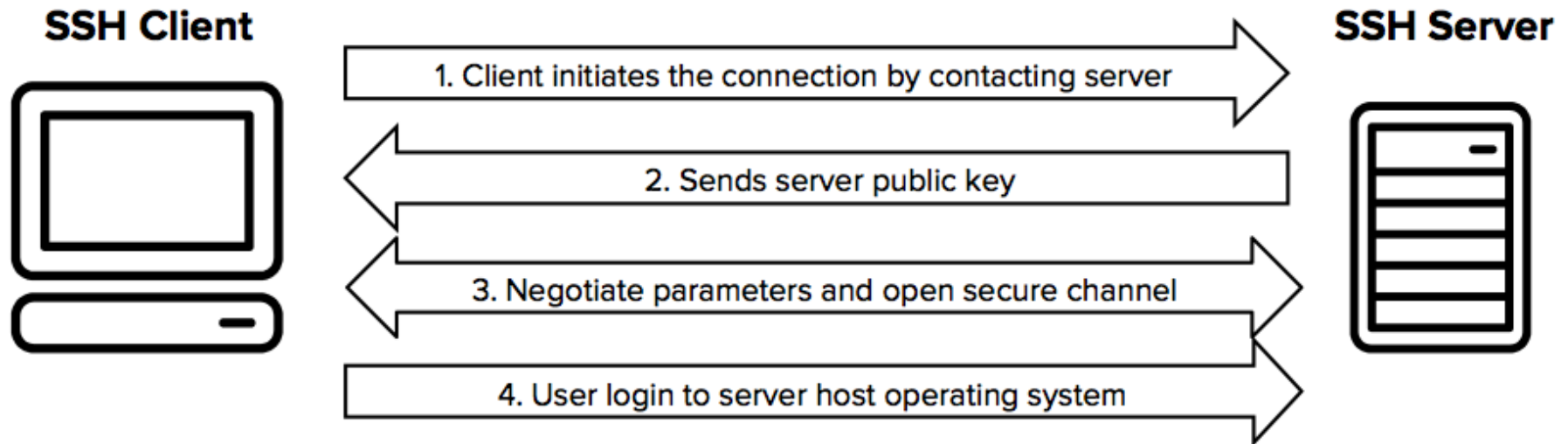


Gorivero / Wikimedia Commons / CC-BY-SA-3.0

Secure Shell

- Cryptographic network protocol
- Several implementations inc. OpenSSH
- Works over TCP/IP (Ports and IP addresses used)
- Provides Session and Presentation facilities of OSI
- Client-Server architecture
- Provides secure channels over unsecured networks

SSH protocol



- The client runs a userspace program “ssh”, which connects to the server port 22
- The server runs a service or daemon “sshd”, which is listening on port 22
- Both have configuration files, see the manual

SSH function

- Establish a client-server session
- Run shell commands on the server
- Pipeline data both ways
- Port forwarding

Intended use

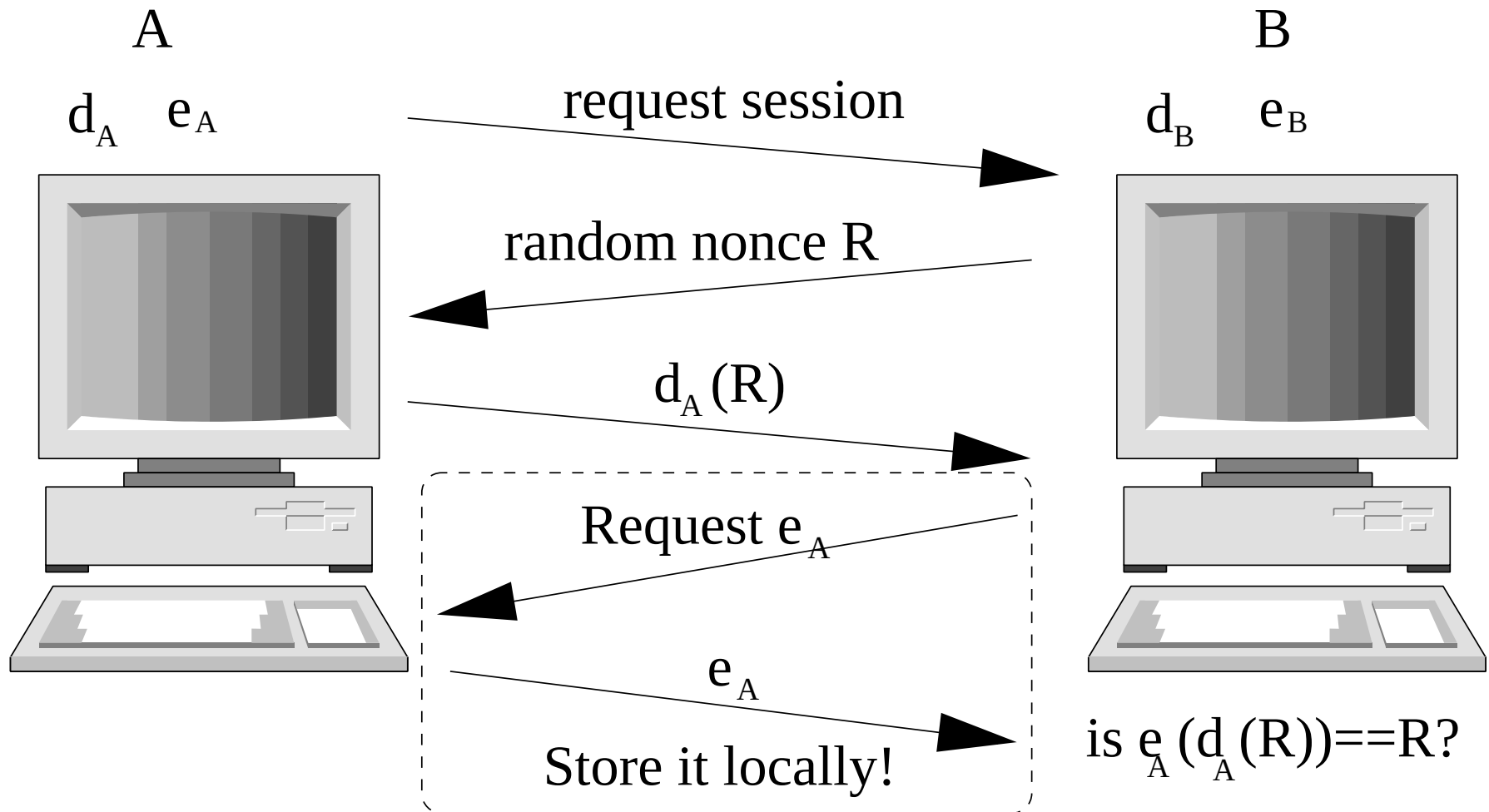
- providing secure access for users and automated processes
- interactive and automated file transfers
- issuing remote commands
- managing network infrastructure and other mission-critical system components

[<https://www.ssh.com/ssh/protocol/>]

Public Key session authentication

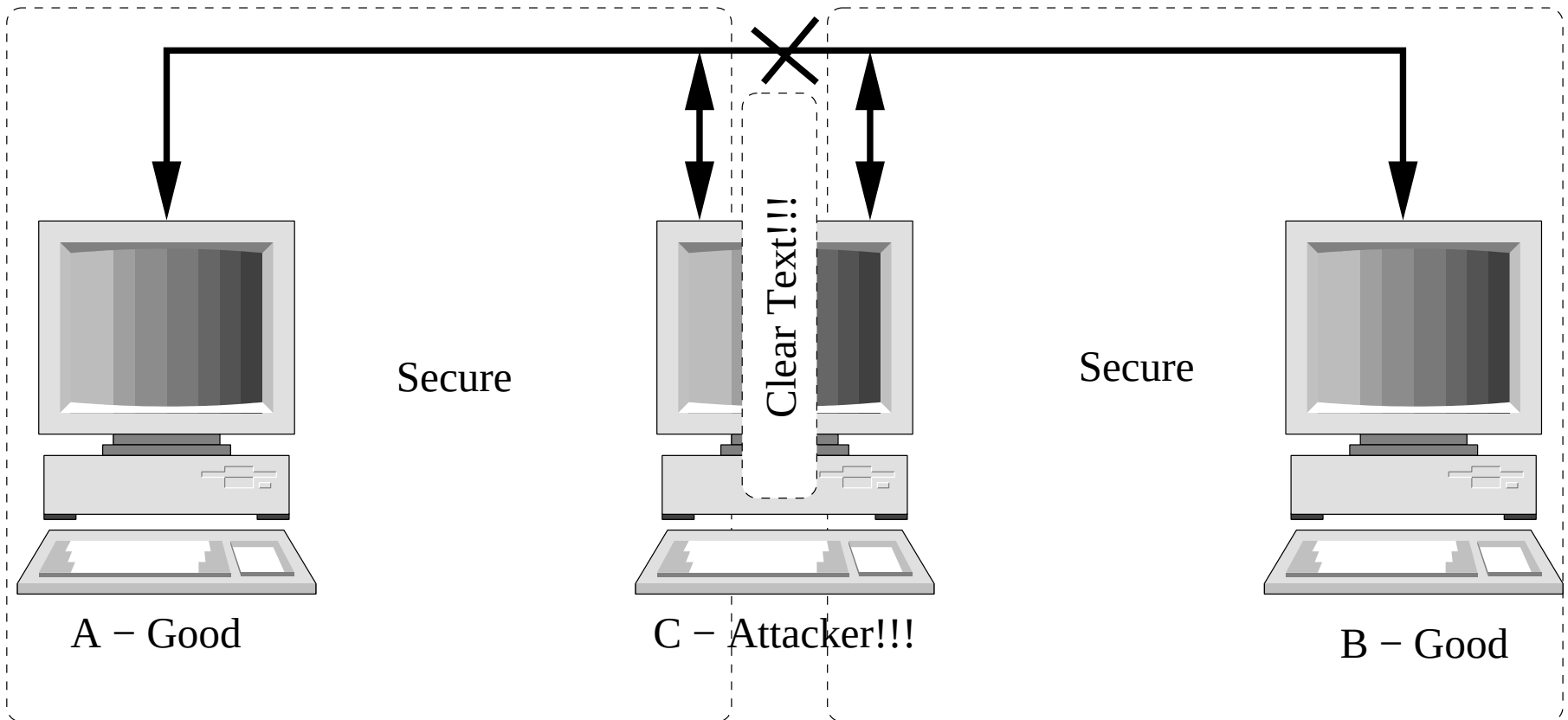
- Client: a private key
- Server: a collection of public keys from the clients
- Any client with a copy of the private key can login
- Transfer of the client's public key to the
 - possible “man in the middle attack”
 - password authentication
 - verification by hash sent through a redundant channel (e.g. telephone)

Public key authentication



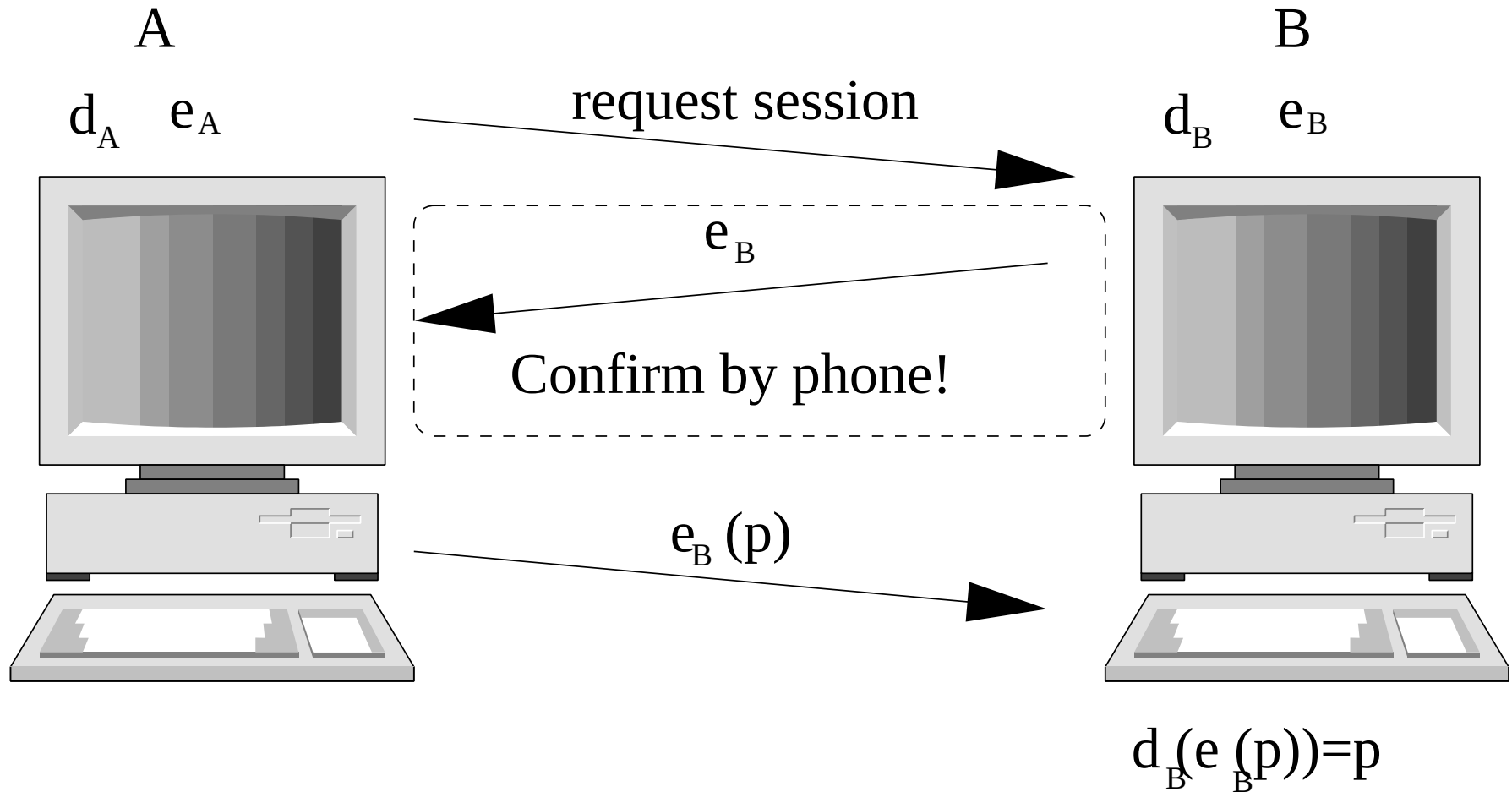
- What can go wrong? – discussion, e_A distribution
- Also used in secure password authentication

Man in the middle threat



- The attacker starts a session on the server by using the credentials of the client
- Relaying all traffic between A and B

Password authentication



SSH protocol stack

SSH User Authentication Protocol Authenticates the client-side user to the server.	SSH Connection Protocol Multiplexes the encrypted tunnel into several logical channels
SSH Transport Layer Protocol Provides server authentication, confidentiality and integrity. It may optionally compress the data. Public key cryptography for session authentication Symmetric key cryptography for data transfer	
TCP	
IP	

Where are the keys stored?

- Host key for the server
 - `/etc/ssh/ssh_host_rsa_key`
 - `/etc/ssh/ssh_host_rsa_key.pub`
 - a couple of additional pairs for different algorithms
- User keys
 - `~/.ssh/id_rsa` #user private key
 - `~/.ssh/id_rsa.pub` #user public key
 - `~/.ssh/known_hosts` #to avoid “fingerprint” confirmation
 - `~/.ssh/authorized_keys` #for remote users automatically connecting
- Password – the user password on the host

Generating SSH keys

- Create a private-public key pair

```
# ssh-keygen
```

Generating public/private rsa key pair.

Enter file in which to save the key
(/home/ylo/.ssh/id_rsa): mykey

Enter passphrase (empty for no passphrase):

Enter same passphrase again:

Your identification has been saved in mykey.

Your public key has been saved in mykey.pub.

The key fingerprint is:

SHA256:GKW7yzA1J1qkr1Cr9MhUwAbHbF2...

- Copy the user key to a remote host

```
# ssh-copy-id -i ~/.ssh/mykey user@hostB
```

SSH pipelines

- SSH starts a process at the remote side and forwards the stdin to it
 - `cat ~/fifoA | ssh user@hostB cat ">" ~/fifoB`
- SSH starts a process at the remote side and grabs its stdout
 - `ssh user@hostB cat ~/fifoB | cat > fifoA`
- In and Out pipelines can be combined.
- Chaining such connections by copying data between the pipes on the same host
 - `cat fifo1A > fifo2A`
- Multiple processes can be started remotely – “agents” for analytics, administration, etc.

M2M with SSH pipelines

Topology

- Star, with a central server
- Several servers
- Ad-hoc

Connectors

- Application specific

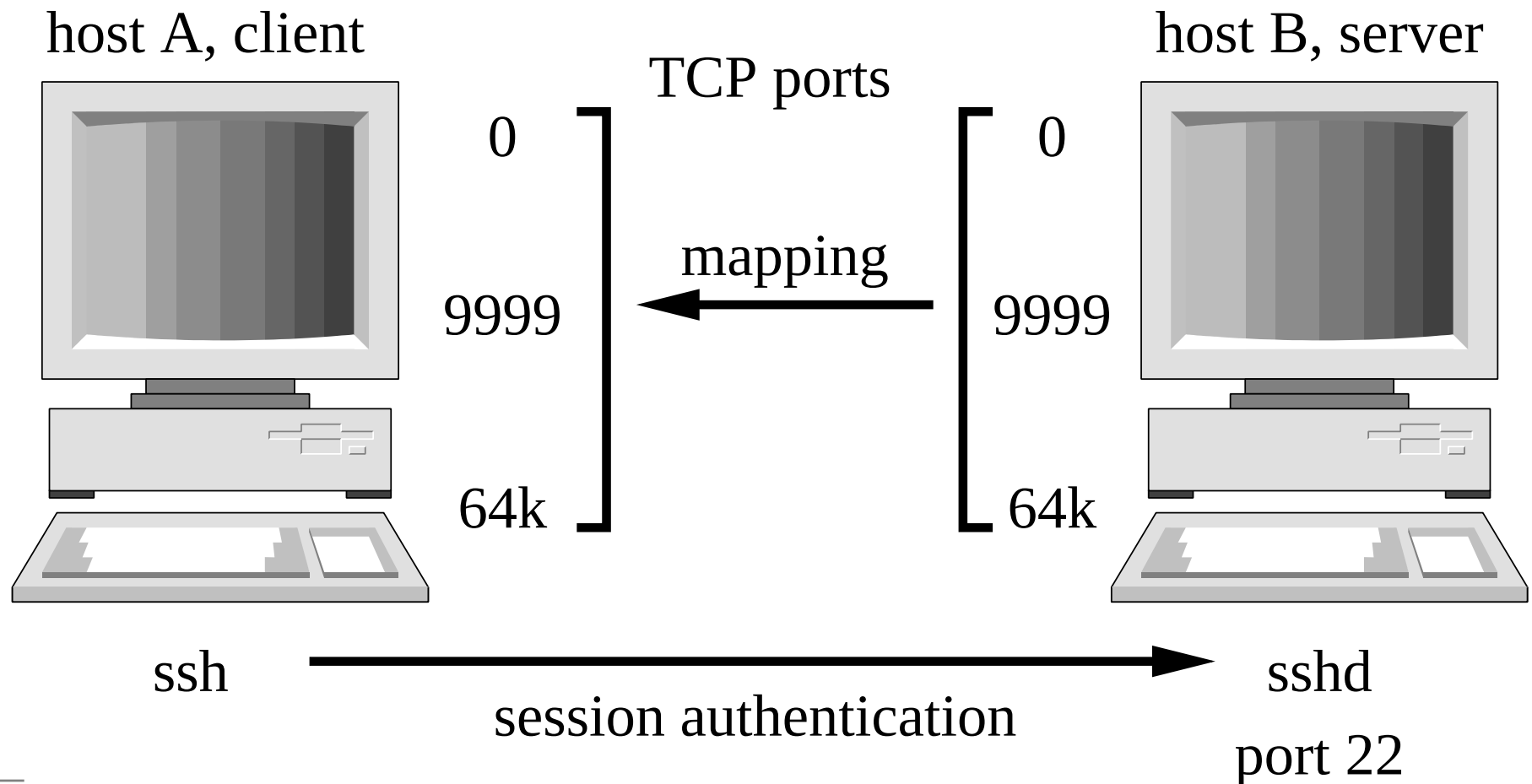
Security

- Flat, no separation between the service provider and user confidentiality at the platform level

SSH port forwarding, local port

```
ssh -L 9999:localhost:9999 user@hostB
```

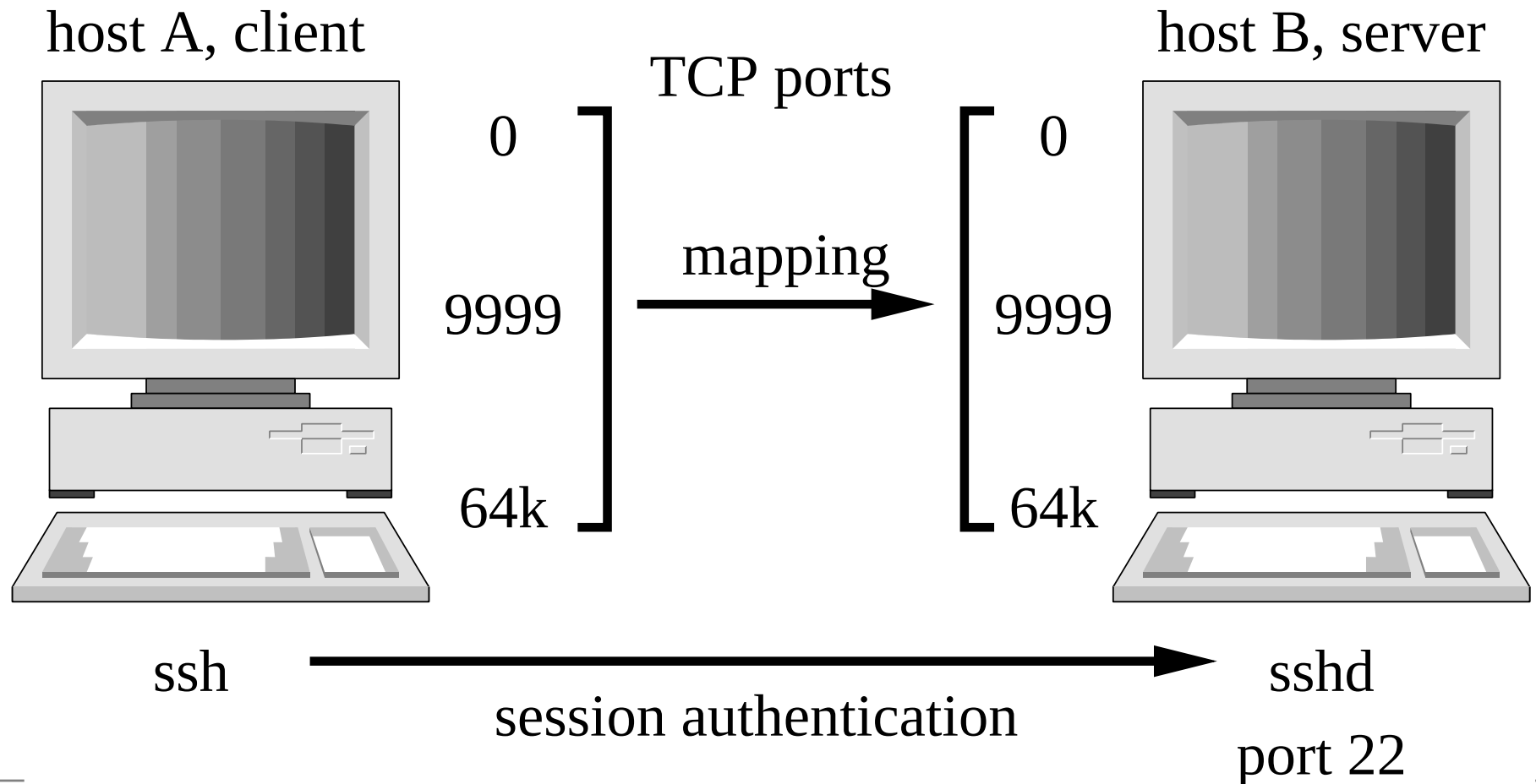
Here “localhost” is an alias for hostB (destination), but can be any other host, e.g. hostC



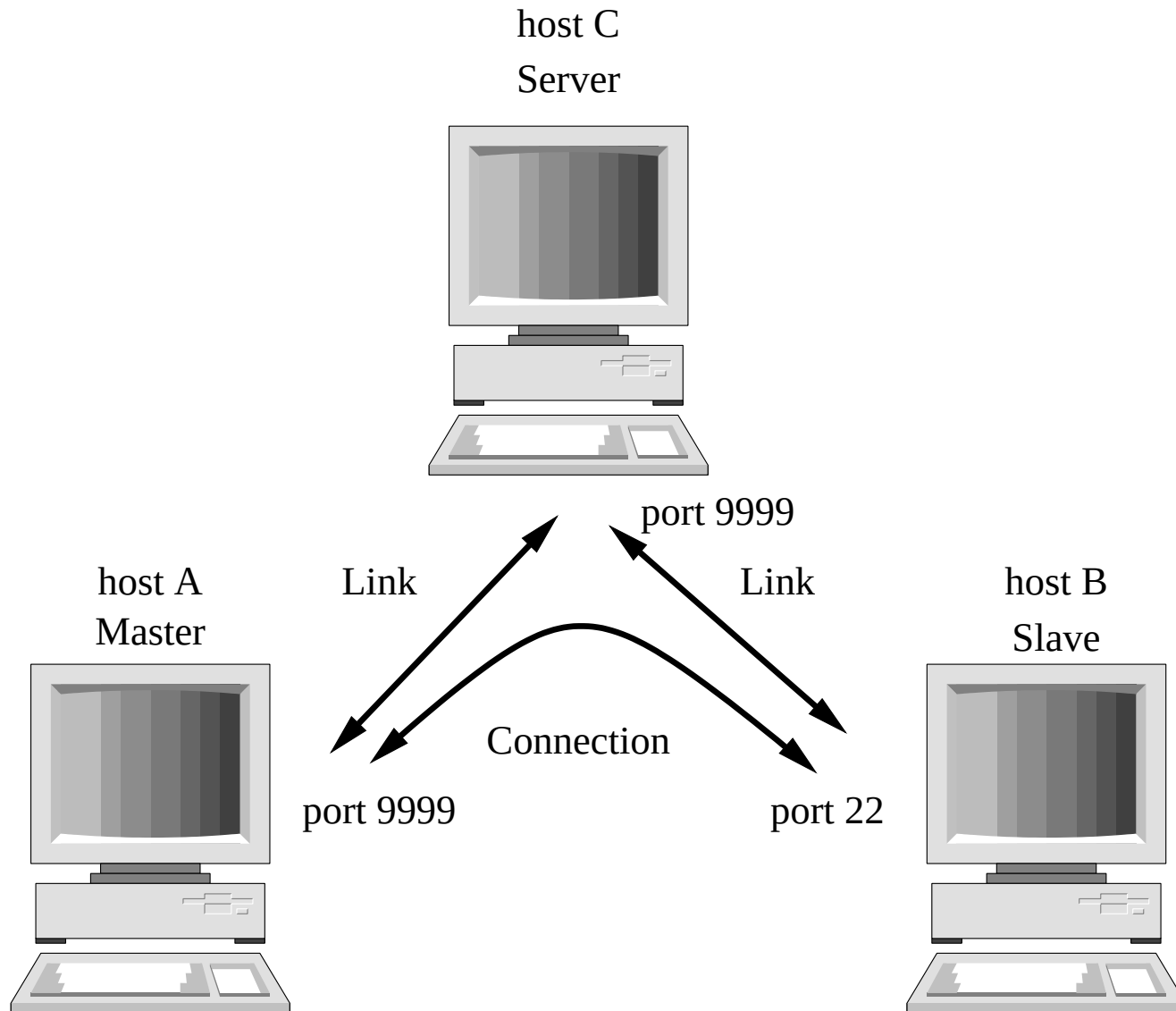
SSH port forwarding, remote port

```
ssh -R 9999:localhost:9999 user@hostB
```

Here “localhost” is an alias for hostA (source), but can be any other host, e.g. hostC



M2M with SSH port forwarding



Links

- Session established from Master/Slave to Server
 - Master/Slave can be on different private networks with NAT gateways
 - Link authentication credentials are known to Server
 - Data are visible to Server as clear text
- `hostA# ssh -L 9999:localhost:9999 user@hostC`
- `hostB# ssh -R 9999:localhost:22 user@hostC`
 - only root can forward privileged ports, i.e. port 22

Connections

- Session established from Master to Slave through the links
 - Master is talking to it's own port (e.g. 9999), which is forwarded through the links from Slave
 - Connection authentication credentials are not known to Server
 - Data are not presented to Server in clear text
- host A# ssh -p 9999 localhost
 - Agnostic of the links as M2M should be

Using such connections

- API – a set of fifo files in Master/Slave mapped into each other
 - `cat master_fifo | ssh -p 9999 localhost cat ">" slave_fifo`
 - `ssh -p 9999 localhost cat "<" slave_fifo > master_fifo`
- limitation – one bidirectional connection per ssh process instance
- many ssh sessions are possible on the same port
- many remote processes can be started to act as Master's "agents"

SSH M2M quality

Advantages

- Established platform, focus on security, two-level security
- Easy to deploy at small scale
- Independent from the service provider, privacy
- An arbitrary topology with multiple Servers, as opposed to star topology of DeviceHive, Google, Amazon, etc. M2M solutions

Disadvantages

- No browser interface, no web tools for user interface
- Needs a prior planning/design
- Analytics have to be manually constructed with agents